

TASK 3 — Selenium Page Object Model in Python Behave

1. Experiment Title

Selenium Page Object Model (POM) Using Python and Behave

2. Problem Statement

Implement a Selenium-based web automation framework using the **Page Object Model (POM)** design pattern with Python and Behave BDD. The login page elements and actions should be separated into a dedicated page class to improve code organization and reusability.

3. Objective

- To implement the **Page Object Model** using Python and Behave.
 - To separate page elements and actions from step definitions.
 - To improve code reusability and maintainability.
 - To automate the SauceDemo login functionality using POM.
-

4. Tools, Software and Concepts Used

Tools & Software

- Python
- Behave
- Selenium WebDriver
- Google Chrome
- PyCharm / VS Code

Concepts Used

- Behaviour-Driven Development (BDD)
- Page Object Model (POM)
- Gherkin

- Selenium WebDriver
 - Python Classes
 - Web Element Locators
 - Reusable Methods
 - Assertions
-

5. Source Code / Implementation Steps

Project Structure

Assignment_3_Behave_POM

```
|
└─ features
    |
    └─ login.feature
        |
        └─ pages
            | └─ login_page.py
            |
            └─ steps
                └─ login_steps.py
```

login.feature

Feature: Login using Page Object Model

Scenario: Successful login using POM

Given I open the SauceDemo login page

When I login with valid credentials

Then I should be logged in successfully

login_page.py

```
from selenium.webdriver.common.by import By
```

```
class LoginPage:
```

```
    USERNAME = (By.ID, "user-name")
```

```
    PASSWORD = (By.NAME, "password")
```

```
    LOGIN_BUTTON = (By.XPATH, "//input[@type='submit']")
```

```
    def __init__(self, driver):
```

```
        self.driver = driver
```

```
    def enter_username(self, username):
```

```
        self.driver.find_element(*self.USERNAME).send_keys(username)
```

```
    def enter_password(self, password):
```

```
        self.driver.find_element(*self.PASSWORD).send_keys(password)
```

```
    def click_login(self):
```

```
        self.driver.find_element(*self.LOGIN_BUTTON).click()
```

```
    def login(self, username, password):
```

```
        self.enter_username(username)
```

```
        self.enter_password(password)
```

```
        self.click_login()
```

login_steps.py

```

from behave import given, when, then

from selenium import webdriver

from features.pages.login_page import LoginPage

@given("I open the SauceDemo login page")
def step_open_login_page(context):
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()
    context.driver.get("https://www.saucedemo.com/")
    context.login_page = LoginPage(context.driver)

@when("I login with valid credentials")
def step_login(context):
    context.login_page.login("standard_user", "secret_sauce")

@then("I should be logged in successfully")
def step_verify_login(context):
    assert "/inventory.html" in context.driver.current_url
    print("Login successful using Page Object Model!")
    context.driver.quit()

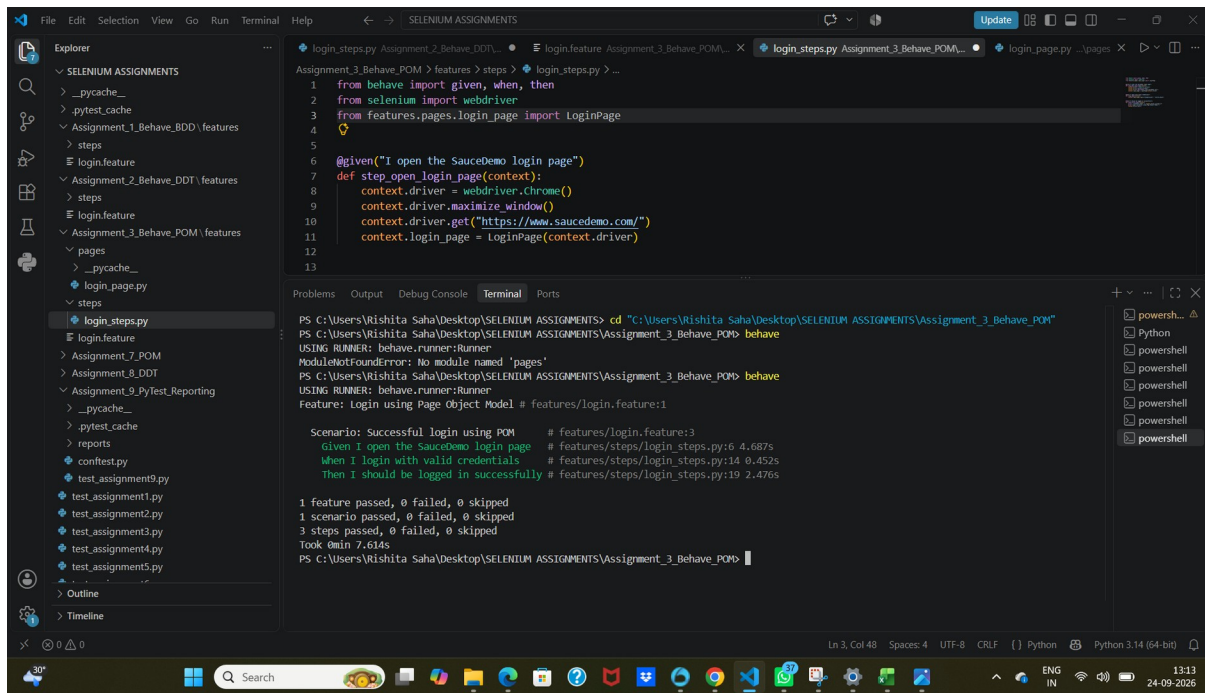
```

Execution

From the Assignment_3_Behave_POM folder:

behave

6. Output with Clearly Labelled Screenshots



Successful Behave Execution

Show the terminal after:

behave

The execution should show:

1 feature passed

1 scenario passed

3 steps passed

7. Result

The SauceDemo login functionality was successfully automated using **Selenium, Python, Behave, and the Page Object Model**. The page elements and login operations were encapsulated inside the LoginPage class.

8. Observation

- Page locators are maintained separately from the step definitions.
- The LoginPage class provides reusable methods for entering credentials and performing login.

- Step definitions become simpler because they interact with page methods instead of directly locating every element.
- The POM structure improves organization and maintainability.

9. Conclusion

This experiment demonstrated the implementation of the **Page Object Model with Selenium Python and Behave BDD**. Separating page-specific elements and actions into a page class makes the automation framework more structured, reusable, and easier to maintain.